UTEC
Universidad Tecnológica

Patrón Proxy

Patrón Proxy: Control de Acceso y Representaciones Virtuales

Descripción General

El patrón Proxy es un patrón estructural que actúa como un intermediario o "sustituto" de otro objeto para controlar su acceso o añadir funcionalidades adicionales. El objetivo principal de este patrón es proporcionar un objeto que controle el acceso a otro, permitiendo realizar operaciones adicionales antes o después de la interacción con el objeto real.

El patrón Proxy se utiliza en situaciones donde:

- **Control de acceso**: Se necesita restringir o controlar el acceso a un objeto.
- **Carga diferida o "lazy loading"**: Se quiere retrasar la creación de un objeto costoso hasta que sea necesario.
- **Representación remota**: Se requiere representar un objeto que reside en un espacio de memoria diferente, como un servidor remoto.
- **Registro de acceso**: Se necesita registrar o auditar el acceso a un objeto.

Tipos de Proxy

1. **Proxy de Protección**: Controla el acceso a un objeto, proporcionando diferentes niveles de permisos.
2. **Proxy Remoto**: Representa un objeto que reside en un espacio de memoria diferente, típicamente en un servidor remoto.
3. **Proxy Virtual**: Retarda la creación de un objeto costoso hasta que se necesite.
4. **Proxy Caché**: Almacena en caché los resultados de operaciones costosas para mejorar el rendimiento.
5. **Proxy de Registro**: Proporciona capacidades de registro para auditar o monitorizar las operaciones realizadas sobre un objeto.

Ventajas

- **Control de acceso centralizado**: El Proxy actúa como un controlador único para el acceso al objeto real.
- **Optimización del rendimiento**: En el caso de Proxy Virtual o Caché, mejora el rendimiento al evitar la creación innecesaria de objetos.
- **Desacoplamiento**: Aísla el cliente del objeto real, lo que puede facilitar cambios en la implementación interna sin afectar a los clientes.

UTEC
Universidad Tecnológica

Desventajas

- **Complejidad añadida**: Introducir un Proxy puede añadir complejidad al diseño y a la implementación del sistema.
- **Posible sobrecarga**: Si no se utiliza adecuadamente, el Proxy puede introducir una sobrecarga innecesaria en la ejecución de operaciones.

Ejemplo en Java

Supongamos que tenemos un sistema de imágenes, donde cargar imágenes grandes desde el disco puede ser un proceso costoso en términos de tiempo. Usaremos un **Proxy Virtual** para cargar la imagen solo cuando realmente sea necesario.

```
// Interfaz común
interface Imagen {
    void mostrar();
}

// Implementación del objeto real
class ImagenReal implements Imagen {
    private String nombreArchivo;

    public ImagenReal(String nombreArchivo) {
        this.nombreArchivo = nombreArchivo;
        cargarImagenDesdeDisco();
    }

    private void cargarImagenDesdeDisco() {
        System.out.println("Cargando " + nombreArchivo);
    }

    @Override
    public void mostrar() {
        System.out.println("Mostrando " + nombreArchivo);
    }
}

// Proxy para ImagenReal
class ImagenProxy implements Imagen {
    private String nombreArchivo;
    private ImagenReal imagenReal;
```

UTEC
Universidad Tecnológica

```
    public ImagenProxy(String nombreArchivo) {
        this.nombreArchivo = nombreArchivo;
    }

    @Override
    public void mostrar() {
        if (imagenReal == null) {
            imagenReal = new ImagenReal(nombreArchivo);
        }
        imagenReal.mostrar();
    }
}

// Uso del patrón Proxy
public class ProxyPatternExample {
    public static void main(String[] args) {
        Imagen imagen = new ImagenProxy("imagen_muy_grande.jpg");

        // La imagen se cargará desde el disco la primera vez que se
muestre
        imagen.mostrar();

        // La imagen ya está cargada, no se vuelve a cargar desde el
disco
        imagen.mostrar();
    }
}
```

Explicación del Ejemplo

En este ejemplo, `Imagen` es una interfaz que define el método `mostrar()`. La clase `ImagenReal` implementa esta interfaz y representa el objeto real, que carga la imagen desde el disco cuando se crea. La clase `ImagenProxy` también implementa `Imagen`, pero retrasa la carga de la imagen hasta que se llame al método `mostrar()` por primera vez. De esta manera, si el cliente no necesita mostrar la imagen, el tiempo de carga se ahorra.

- **Proxy Virtual**: El `ImagenProxy` actúa como un Proxy Virtual que aplaza la carga de una imagen grande hasta que sea necesario.

UTEC
Universidad Tecnológica

Otros Usos Comunes del Patrón Proxy

- **Proxy de Protección**: Controlar quién puede acceder a ciertos recursos, como en un sistema de autenticación.
- **Proxy Remoto**: Trabajar con objetos que están en un servidor remoto, como en sistemas distribuidos.
- **Proxy de Registro**: Registrar o auditar las operaciones realizadas sobre un objeto, por ejemplo, en un sistema de monitoreo.

Conclusión

El patrón Proxy es una herramienta poderosa en el diseño de software, permitiendo a los desarrolladores controlar el acceso a objetos, optimizar el rendimiento mediante la carga diferida, y simplificar la interacción con objetos que pueden estar en un entorno remoto o ser costosos de crear. Es fundamental en situaciones donde la eficiencia, la seguridad, o la simplicidad son importantes, y proporciona una capa adicional de control y flexibilidad.